

Practical-6 Output

Initialization:

This section defines the number of processes and resources and initializes the allocation matrix, maximum resource matrix, and available resource vector.



```
GNU nano 7.2 banker.c *
#include <stdio.h>

int main()
{
    int i, j, k;
    int processes = 5;
    int resources = 4;

    int allocation[5][4] = {
        {0,0,1,4},
        {0,6,3,2},
        {0,0,1,2},
        {1,0,0,0},
        {1,3,5,4}
    };

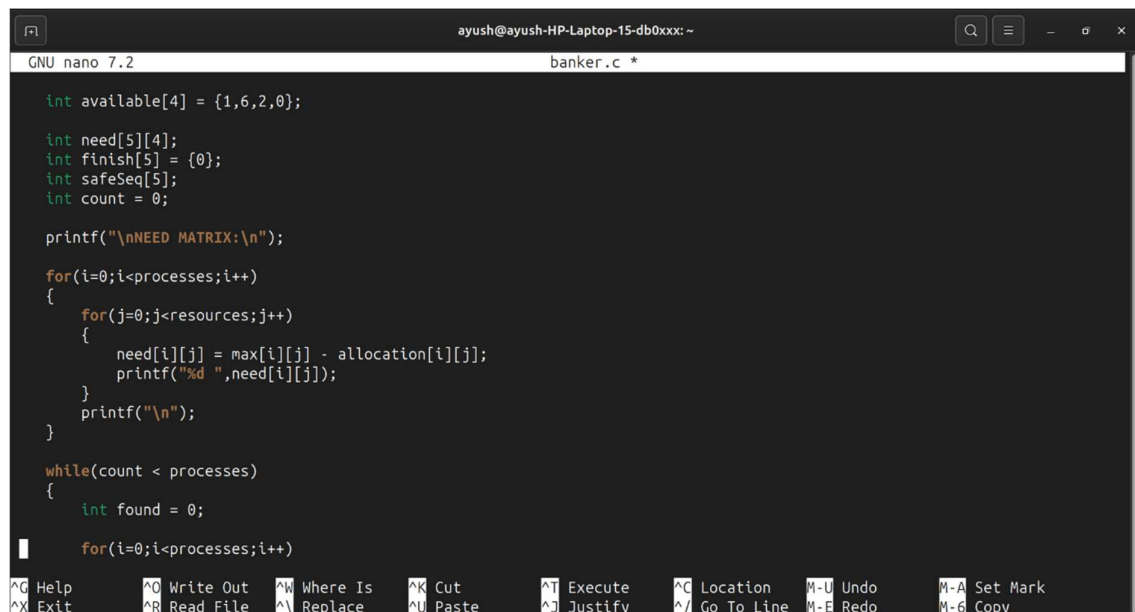
    int max[5][4] = {
        {0,6,5,6},
        {0,6,5,2},
        {0,0,1,2},
        {1,7,5,0},
        {2,3,5,6}
    };

    int available[4] = {1,6,2,0};

    ^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
    ^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-G Copy
```

Calculation:

This section computes the Need Matrix by subtracting the allocated resources from the maximum required resources for each process.



```
GNU nano 7.2 banker.c *

    int available[4] = {1,6,2,0};

    int need[5][4];
    int finish[5] = {0};
    int safeSeq[5];
    int count = 0;

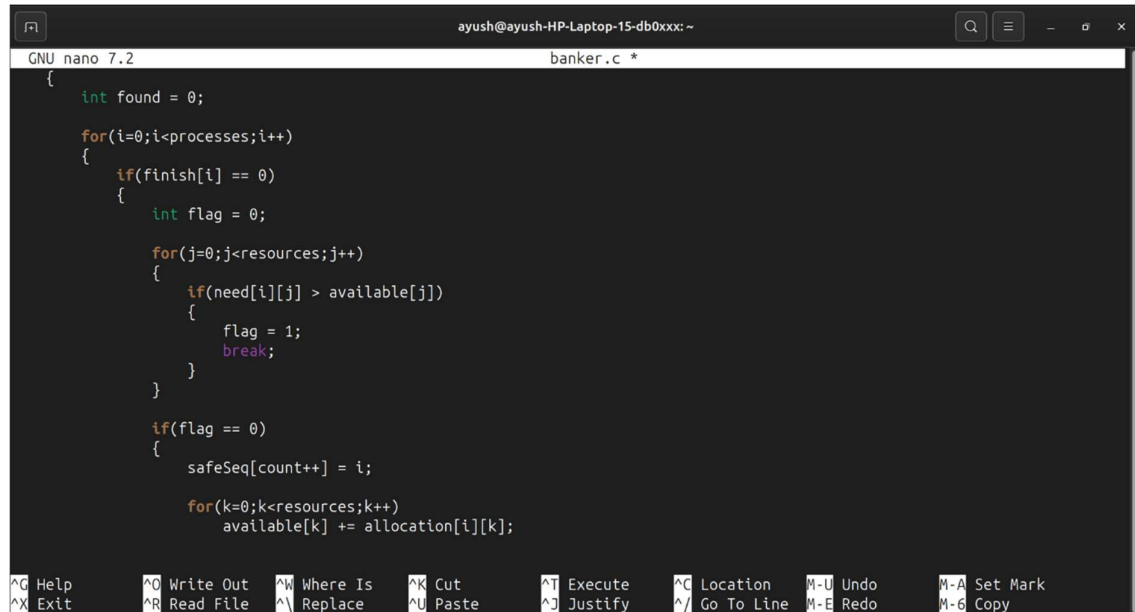
    printf("\nNEED MATRIX:\n");
    for(i=0;i<processes;i++)
    {
        for(j=0;j<resources;j++)
        {
            need[i][j] = max[i][j] - allocation[i][j];
            printf("%d ",need[i][j]);
        }
        printf("\n");
    }

    while(count < processes)
    {
        int found = 0;

        for(i=0;i<processes;i++)
```

Verification:

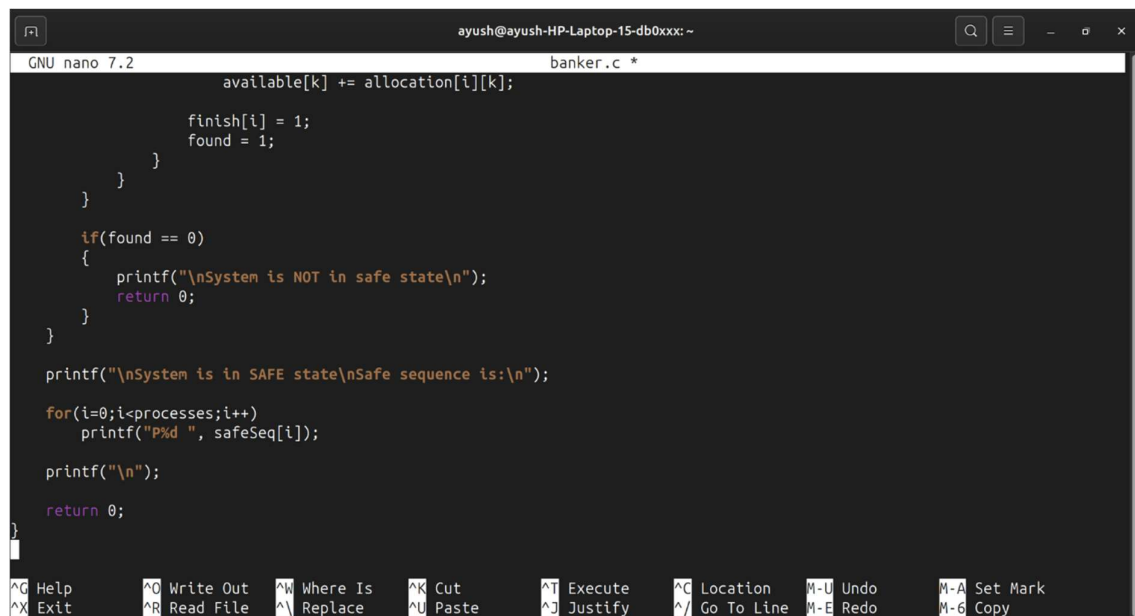
This section checks whether each process can execute by comparing its resource needs with the currently available resources to determine system safety.



```
GNU nano 7.2 banker.c *
{
    int found = 0;
    for(i=0;i<processes;i++)
    {
        if(finish[i] == 0)
        {
            int flag = 0;
            for(j=0;j<resources;j++)
            {
                if(need[i][j] > available[j])
                {
                    flag = 1;
                    break;
                }
            }
            if(flag == 0)
            {
                safeSeq[count++] = i;
                for(k=0;k<resources;k++)
                    available[k] += allocation[i][k];
            }
        }
    }
}
```

Execution:

This section determines whether the system is in a safe state and prints the safe sequence of process execution using Banker's Algorithm.



```
GNU nano 7.2 banker.c *
        available[k] += allocation[i][k];
        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
{
    printf("\nSystem is NOT in safe state\n");
    return 0;
}

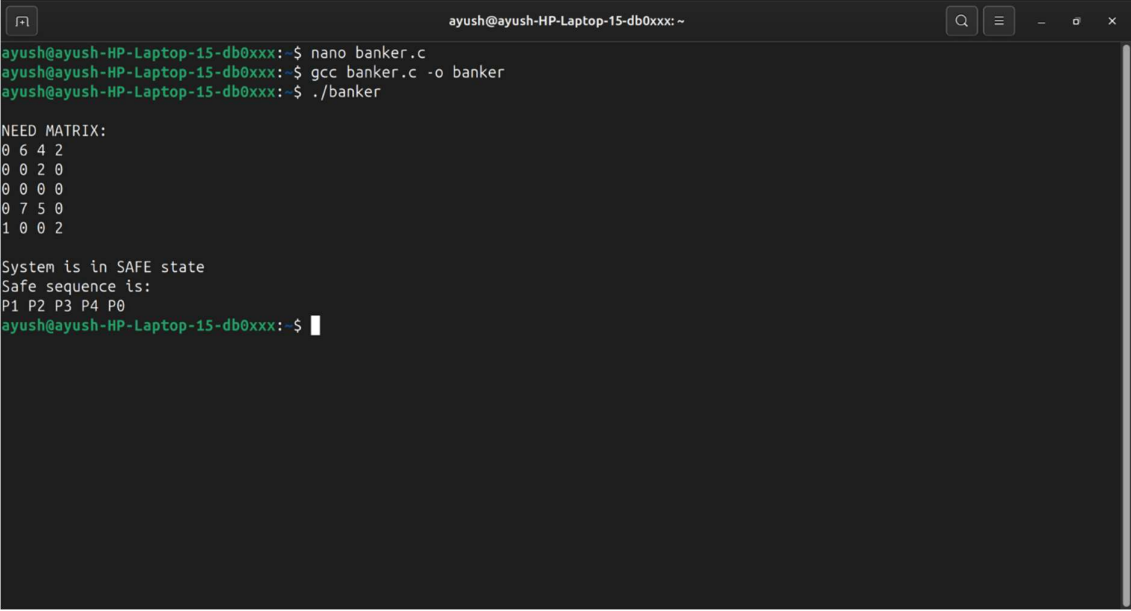
printf("\nSystem is in SAFE state\nSafe sequence is:\n");
for(i=0;i<processes;i++)
    printf("P%d ", safeSeq[i]);

printf("\n");

return 0;
}
```

Output:

This section shows the execution result of the Banker's Algorithm program, displaying the Need Matrix, confirming that the system is in a safe state, and printing the safe sequence of processes.



```
ayush@ayush-HP-Laptop-15-db0xxx: ~  
ayush@ayush-HP-Laptop-15-db0xxx:~$ nano banker.c  
ayush@ayush-HP-Laptop-15-db0xxx:~$ gcc banker.c -o banker  
ayush@ayush-HP-Laptop-15-db0xxx:~$ ./banker  
  
NEED MATRIX:  
0 6 4 2  
0 0 2 0  
0 0 0 0  
0 7 5 0  
1 0 0 2  
  
System is in SAFE state  
Safe sequence is:  
P1 P2 P3 P4 P0  
ayush@ayush-HP-Laptop-15-db0xxx:~$
```